

Notes for fall 2014 Exam 2

1. 29, 7, 2, 18, 17, 3, 15, 30, 9, 6

2.1. Can reduce fragmentation

2.2. False only for $n > n_0$ for some n_0 . For small n the lesser terms may be significant.

2.3. $E = I + 2n$.

2.4. $\text{floor}(\log_2 n) + 1$

2.5. $O(n)$

2.6. $\text{Rear} = (\text{Count} + \text{Front}) \% N$.

2.7. right child at $2i + 1$ if $2i + 1 \leq n$

2.8. True

3.1. Single left at 8. 18 is new root.

3.2. Double right at 21. 15 is new root

3.3. No change to shape of tree.

3.4. Increase by one level with H as new root.

3.5. 29 ends up in location of 19, and 19 and 3 pushed down.

3.6. 43, 13, 37, 8, 10, 1, 32, 5, 4

4.1. A binary search requires jumping to the middle of the list. Moving to the middle in a linked list can be $O(n)$ which is more costly than accessing an array element.

4.2. $O(n \log n)$

4.3. Considering all n packets

inserts are $O(n^2)$ because an insertion sort

removes are $O(n)$ because removing head is $O(1)$

4.4. A heap. Each insert and remove is $O(\log n)$. The overall complexity for both phases is $O(n \log n)$ which is less than $O(n^2)$ for 4.3.

5. Many solutions are possible. Here is one.

```
int tree_median(binary_tree_t *T) {
    int median;
    median_rec(T->root, (T->count/2)+1, 0, &median);
    return median;
}

int median_rec(tree_node_t *N, int target, int count, int *median) {
    if (N == NULL) return 0;
    int left_cnt = median_rec(N->left, target, count, median);
    if (count + 1 + left_cnt == target)
        *median = N->key;
    int right_cnt = median_rec(N->right, target, count+1+left_cnt, median);
    return 1 + left_cnt + right_cnt;
}
```